

Karen AI Chat System - New Architecture Guide

Overview

This document provides a detailed explanation of Karen AI's new chat system architecture, focusing on the request flow, component roles, and how the system works after the three-phase refactoring.

Architecture Principles

The new architecture is built on several key principles:

- Single Source of Truth:** ChatOrchestrator is the absolute authority for the chat response lifecycle.
- Clear Separation of Concerns:** Each component has a well-defined responsibility.
- Transactional Integrity:** Memory operations are transactional to response generation.
- Orchestrated Fallbacks:** All fallback decisions are centralized under ChatOrchestrator control.
- Observability:** Comprehensive logging and metrics at every stage.

Core Components

1. ChatOrchestrator

Location: `src/ai_karen_engine/chat/chat_orchestrator.py`

Role: The central coordinator and single source of truth for the chat response lifecycle.

Key Responsibilities:

- Coordinate all aspects of chat message processing
- Manage the processing pipeline with retry logic
- Orchestrate memory operations (recall and writeback)
- Control fallback mechanisms through FallbackRouter
- Provide comprehensive error handling and logging
- Collect metrics and monitor system health

Key Methods:

- `process_message()`: Main entry point for processing chat requests
- `_process_message_internal()`: Core message processing logic
- `_orchestrate_post_response_memory_writeback()`: Transactional memory writeback
- `_retrieve_context()`: Memory context retrieval

2. FallbackRouter

Location: `src/ai_karen_engine/chat/chat_orchestrator.py:69`

Role: Centralizes all fallback decisions under ChatOrchestrator control.

Key Responsibilities:

- Manage the fallback chain of LLM providers
- Make intelligent fallback decisions based on system state
- Activate degraded mode when all providers fail
- Track fallback metrics for monitoring

Key Methods:

- `create_fallback_context()`: Create context for tracking fallback decisions
- `record_fallback_attempt()`: Record a fallback attempt in the context
- `should_enter_degraded_mode()`: Determine if system should enter degraded mode
- `activate_degraded_mode()`: Activate degraded mode through the orchestrator's governance

3. Route Handler (Thin Ingress Layer)

Location: `src/ai_karen_engine/api_routes/copilot_routes.py`

Role: Handle HTTP-specific concerns and delegate to ChatOrchestrator.

Key Responsibilities:

- Request validation and normalization
- Authentication and authorization checks
- Creating ChatRequest objects
- Delegating to ChatOrchestrator
- Formatting ChatResponse objects as HTTP responses

Key Endpoint:

- `/assist`: Main chat processing endpoint

4. Memory Operations

Role: Handle memory recall and writeback under ChatOrchestrator control.

Key Responsibilities:

- **Pre-Response Recall:** Retrieve relevant context before response generation
- **Post-Response Writeback:** Persist successful exchanges to memory
- **Transactional Integrity:** Ensure memory writes only happen after successful responses

Key Methods:

- `_retrieve_context()`: Retrieve relevant context for the message
- `_orchestrate_post_response_memory_writeback()`: Orchestrate post-response memory writeback

Request Flow

Step-by-Step Request Processing

1. HTTP Request Arrival

- A client sends a request to the `/assist` endpoint
- The request includes the user message, user ID, and optional metadata

2. Request Processing in Route Handler

```
@router.post("/assist", response_model=AssistResponse)
async def copilot_assist(
    request: AssistRequest,
    http_request: Request,
):
```

- The route handler validates and normalizes the request
- Authentication and authorization checks are performed
- A correlation ID is extracted or generated for tracking

3. ChatRequest Creation

```
chat_request = ChatRequest(
    message=message,
    user_id=user_id,
    conversation_id=session_id,
    session_id=session_id,
    stream=False,
    include_context=True,
    metadata={
        "source": "copilot",
        "org_id": org_id,
        "platform": "copilot",
        # ... additional metadata
    },
)
```

- The route handler creates a `ChatRequest` object
- All relevant information is encapsulated in the request object

4. Delegation to ChatOrchestrator

```
response = await chat_orchestrator.process_message(chat_request)
```

- The route handler delegates processing to `ChatOrchestrator`
- The `ChatRequest` object is passed to the orchestrator

5. Processing Context Creation

```
context = ProcessingContext(
    user_id=request.user_id,
    conversation_id=request.conversation_id,
    session_id=request.session_id,
    metadata=request.metadata
)
```

- ChatOrchestrator creates a processing context
- The context includes correlation ID, user information, and metadata

6. Processing Pipeline Execution

- ChatOrchestrator executes the processing pipeline with retry logic
- The pipeline includes NLP processing, memory operations, and LLM integration
- Each step is monitored and logged

7. Memory Operations

- **Step 6: PRE-RESPONSE MEMORY RECALL**

```
raw_context = await self._retrieve_context(
    embeddings,
    parsed_message,
    request.user_id,
    request.conversation_id
)
```

- Relevant context is retrieved from memory
- The context includes memories, entities, and user preferences

- **Step 9: POST-RESPONSE MEMORY WRITEBACK**

```
writeback_status = await
self._orchestrate_post_response_memory_writeback(
    request=request,
    context=context,
    result=result,
)
```

- Successful exchanges are persisted to memory
- Memory writes are transactional to response generation

8. LLM Response Generation

- ChatOrchestrator generates AI responses using the LLM orchestrator
- Fallback mechanisms are managed by the FallbackRouter

- The response includes metadata about the generation process

9. Response Formatting

```
return AssistResponse(  
    answer=answer,  
    structured_content=_json_safe(structured_content),  
    actions=_json_safe(actions),  
    metadata=_json_safe(metadata),  
    correlation_id=correlation_id,  
)
```

- The route handler formats the ChatResponse as an HTTP response
- The response includes the answer, structured content, and metadata

10. HTTP Response Return

- The formatted response is returned to the client
- The response includes correlation ID for tracking

Detailed Processing Pipeline

The ChatOrchestrator's processing pipeline consists of several steps:

1. Message Parsing with spaCy

```
parsed_message = await  
nlp_service_manager.parse_message(request.message)
```

- The user message is parsed using spaCy NLP
- Entities, tokens, and other linguistic features are extracted

2. Instruction Processing

```
extracted_instructions = await  
self.instruction_processor.extract_instructions(  
    request.message, instruction_context  
)
```

- Instructions are extracted from the message
- Active instructions are retrieved for context

3. Embedding Generation

```
embeddings = await
nlp_service_manager.get_embeddings(request.message)
```

- Semantic embeddings are generated for the message
- Embeddings are used for memory similarity search

4. Memory Context Retrieval

```
raw_context = await self._retrieve_context(
    embeddings,
    parsed_message,
    request.user_id,
    request.conversation_id
)
```

- Relevant memories are retrieved based on semantic similarity
- The context includes memories, entities, and user preferences

5. Context Integration

```
integrated_context = await
self.context_integrator.integrate_context(
    raw_context,
    request.message,
    request.user_id,
    request.conversation_id
)
```

- Retrieved context is integrated and enhanced
- The context is prepared for LLM consumption

6. AI Response Generation

```
ai_response, llm_metadata, llm_used_fallback = (
    await self._generate_ai_response_enhanced(
        request.message,
        parsed_message,
        embeddings,
        integrated_context,
        active_instructions,
        context,
    )
)
```

- AI response is generated using the LLM orchestrator
- Fallback mechanisms are managed by the FallbackRouter
- Metadata about the generation process is collected

7. Response Formatting

```
formatted_result = self.output_layer.format_response(ai_response,
formatter_ctx)
ai_response = formatted_result.get("content", ai_response)
```

- The response is formatted using the PrettyOutputLayer
- Formatting metadata is merged into the response metadata

8. Memory Writeback

```
writeback_status = await
self._orchestrate_post_response_memory_writeback(
    request=request,
    context=context,
    result=result,
)
```

- Successful exchanges are persisted to memory
- Memory writes are transactional to response generation

Memory Operations

Pre-Response Memory Recall

Purpose: Gather relevant context BEFORE response generation.

When: Happens in `_process_message_internal()` at Step 6.

Operations:

1. Relevant recall using semantic similarity search
2. Ranked context injection based on combined scores
3. Continuity support through recency scoring
4. Persona/profile grounding through context integration
5. Attachment context merging
6. Instruction integration

Implementation:

```
async def _retrieve_context(
    self,
```

```

        embeddings: List[float],
        parsed_message: ParsedMessage,
        user_id: str,
        conversation_id: str
    ) -> Dict[str, Any]:
        """Retrieve relevant context for the message using
        MemoryProcessor."""
        if not self.memory_processor:
            # Fallback context when memory processor is not available
            return {
                "memories": [],
                "conversation_history": [],
                "user_preferences": {},
                "entities": [{"text": ent[0], "label": ent[1]} for ent in
parsed_message.entities],
                "embedding_similarity_threshold": 0.7,
                "context_summary": "Memory processor not available"
            }

        # Use MemoryProcessor to get relevant context
        memory_context = await self.memory_processor.get_relevant_context(
            embeddings,
            parsed_message,
            user_id,
            conversation_id
        )

        # Convert MemoryContext to dictionary format
        context = {
            "memories": [
                {
                    "id": mem.id,
                    "content": mem.content,
                    "type": mem.memory_type.value,
                    "similarity_score": mem.similarity_score,
                    "recency_score": mem.recency_score,
                    "combined_score": mem.combined_score,
                    "created_at": mem.created_at.isoformat(),
                    "metadata": mem.metadata
                }
                for mem in memory_context.memories
            ],
            "entities": memory_context.entities,
            "preferences": memory_context.preferences,
            "facts": memory_context.facts,
            "relationships": memory_context.relationships,
            "context_summary": memory_context.context_summary,
            "retrieval_time": memory_context.retrieval_time,
            "total_memories_considered":
memory_context.total_memories_considered,
            "embedding_similarity_threshold":
self.memory_processor.similarity_threshold
        }

```

```
return context
```

Post-Response Memory Writeback

Purpose: Persist successful exchanges to memory AFTER response finalization.

When: Happens in `_process_message_internal()` at Step 9.

Operations:

1. Evaluate whether exchange should be retained
2. Summarization or promotion when appropriate
3. Embedding or semantic persistence when appropriate
4. Metadata capture (LLM info, conversation ID, user message)
5. Writeback coordination through memory service
6. Link response to source memory shards for traceability

Implementation:

```
async def _orchestrate_post_response_memory_writeback(
    self,
    *,
    request: ChatRequest,
    context: ProcessingContext,
    result: ProcessingResult,
) -> Dict[str, Any]:
    """
    Orchestrate post-response memory writeback in a transactional
    manner.

    This method is the SINGLE point of control for all memory writeback
    operations.
    Memory writes only occur AFTER successful response generation,
    ensuring
    transactional integrity - no phantom memory writes from failed
    responses.
    """
    # Transactional guard: Only write back if response generation
    succeeded
    if not result.success or not result.response or not request.user_id:
        logger.debug(
            "Skipping memory writeback for %s: response not successful
or empty",
            context.correlation_id
        )
    return {
        "queued": False,
        "linked_shards": 0,
        "reason": "response_not_successful"
```

```

    }

    try:
        from services.memory.internal.memory_writeback import
InteractionType
        from services.memory.unified_memory_service import ContextHit
        from ai_karen_engine.chat.dependencies import get_memory_service

        memory_service = get_memory_service()
        if memory_service is None or not hasattr(memory_service,
"queue_interaction_writeback"):
            logger.warning(
                "Memory service not available for writeback
orchestration: %s",
                context.correlation_id
            )
            return {"queued": False, "linked_shards": 0, "reason":
"memory_service_unavailable"}

        # Step 1: Normalize context hits from retrieved memories
        normalized_hits: List[ContextHit] = []
        raw_memories = result.context.get("memories", []) if
isinstance(result.context, dict) else []
        for item in raw_memories:
            # ... normalization logic ...
            normalized_hits.append(
                ContextHit(
                    id=memory_id,
                    text=text,
                    preview=text[:200],
                    score=float(item.get("combined_score") or
item.get("similarity_score") or 0.0),
                    tags=[
                        str(tag)
                        for tag in (item.get("metadata", {}) or
{}).get("tags", [])
                        if isinstance(tag, str)
                    ],
                    meta=item.get("metadata") if
isinstance(item.get("metadata"), dict) else {},
                    importance=int((item.get("metadata", {}) or
{}).get("importance", 5)),
                    decay_tier=str((item.get("metadata", {}) or
{}).get("decay_tier", "short")),
                    created_at=created_at,
                    updated_at=None,
                    user_id=request.user_id,
                    org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
                )
            )

        # Step 2: Link response to source memory shards

```

```

        shard_links = []
        if normalized_hits and hasattr(memory_service,
"link_response_to_shards"):
            shard_links = await memory_service.link_response_to_shards(
                response_id=context.correlation_id,
                response_content=result.response,
                source_context_hits=normalized_hits,
                user_id=request.user_id,
                org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
                correlation_id=context.correlation_id,
            )

        # Step 3: Queue interaction writeback (the actual memory
persistence)
        writeback_id = await memory_service.queue_interaction_writeback(
            content=result.response,
            interaction_type=InteractionType.COPILOT_RESPONSE,
            user_id=request.user_id,
            org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
            session_id=request.session_id,
            source_shards=shard_links,
            tags=["chat", "response"],
            importance=7,
            metadata={
                "conversation_id": request.conversation_id,
                "user_message": request.message[:1000],
                "llm": result.llm_metadata or {},
                "surface": "chat_orchestrator",
                "orchestrated_by":
"ChatOrchestrator._orchestrate_post_response_memory_writeback",
            },
            correlation_id=context.correlation_id,
        )

        logger.info(
            "Successfully orchestrated memory writeback for %s:
queued=%s, linked_shards=%d",
            context.correlation_id,
            bool(writeback_id),
            len(shard_links)
        )

        return {
            "queued": bool(writeback_id),
            "linked_shards": len(shard_links),
            "writeback_id": str(writeback_id) if writeback_id else None,
            "normalized_hits": len(normalized_hits)
        }
    except Exception as exc:
        logger.warning(
            "Failed to orchestrate memory writeback for %s: %s",

```

```

        context.correlation_id,
        exc,
    )
    return {
        "queued": False,
        "linked_shards": 0,
        "error": str(exc),
        "reason": "writeback_exception"
    }

```

Fallback Mechanisms

FallbackRouter

The FallbackRouter centralizes all fallback decisions under ChatOrchestrator control:

```

class FallbackRouter:
    """
    Orchestrator-controlled fallback routing system.

    This class centralizes all fallback decisions and ensures they
    remain
    within the ChatOrchestrator's governance. No route handler or
    service should independently invoke fallback providers.
    """

    def __init__(self, orchestrator: 'ChatOrchestrator'):
        self.orchestrator = orchestrator
        self.degraded_mode_manager = get_degraded_mode_manager()

        # Load fallback chain from config
        from ai_karen_engine.config.config_manager import
get_fallback_chain
        self.fallback_chain = get_fallback_chain()

        # Fallback metrics
        self._total_fallbacks = 0
        self._degraded_activations = 0
        self._fallback_by_level: Dict[str, int] = {
            "system_default": 0,
            "local": 0,
            "degraded": 0,
        }

```

PROF

Fallback Decision Process

1. **User Preference:** Try the user's preferred provider/model first
2. **System Default:** If user preference fails, try system default providers
3. **Local Fallback:** If system defaults fail, try local models

4. **Degraded Mode:** If all else fails, activate degraded mode

Degraded Mode

Degraded mode is activated when all providers in the fallback chain have been attempted:

```
def should_enter_degraded_mode(
    self,
    context: FallbackContext,
    last_error: Optional[Exception] = None
) -> bool:
    """
    Determine if system should enter degraded mode.

    Degraded mode is triggered when:
    1. All providers in the fallback chain have been attempted
    2. The degraded mode manager is not already active
    3. No local fallback succeeded
    """
    # Check if degraded mode is already active
    if self.degraded_mode_manager.get_status().is_active:
        logger.debug(
            f"Degraded mode already active for {context.correlation_id}"
        )
        return False

    # Check if we've exhausted the fallback chain
    providers_attempted = set(context.providers_attempted)
    fallback_chain_set = set(self.fallback_chain)

    # If we've tried most of the fallback chain without success
    chain_exhausted = len(
        providers_attempted.intersection(fallback_chain_set)
    ) >= len(self.fallback_chain) - 1

    if chain_exhausted and context.attempt_count >= 3:
        logger.warning(
            f"Fallback chain exhausted for {context.correlation_id}. "
            f"Entering degraded mode. Attempted: "
            f"{context.providers_attempted}"
        )
        return True

    return False
```

PROF

Error Handling

Comprehensive Error Handling

The new architecture provides comprehensive error handling at every stage:

1. **Request Validation:** Errors in request validation are caught and returned as HTTP 400 errors
2. **Authentication Errors:** Authentication failures are returned as HTTP 401 errors
3. **Authorization Errors:** Authorization failures are returned as HTTP 403 errors
4. **Processing Errors:** Processing errors are caught and handled gracefully
5. **Memory Errors:** Memory operation errors are logged and handled gracefully
6. **LLM Errors:** LLM provider errors are handled through the fallback mechanism

Error Recovery

The system implements several error recovery mechanisms:

1. **Retry Logic:** The processing pipeline includes retry logic with exponential backoff
2. **Fallback Mechanisms:** The FallbackRouter provides multiple levels of fallback
3. **Degraded Mode:** When all else fails, the system enters degraded mode
4. **Graceful Degradation:** The system continues to provide limited functionality even when some services fail

Monitoring and Observability

Logging

The system provides comprehensive logging at every stage:

1. **Request Logging:** All requests are logged with correlation IDs
2. **Processing Logging:** Each step of the processing pipeline is logged
3. **Error Logging:** All errors are logged with detailed context
4. **Performance Logging:** Processing times and performance metrics are logged

Metrics

The system collects various metrics for monitoring:

1. **Request Metrics:** Total requests, successful requests, failed requests
2. **Performance Metrics:** Processing times, response times
3. **Memory Metrics:** Memory recall times, writeback success rates
4. **Fallback Metrics:** Fallback usage, degraded mode activations

Health Checks

The system provides health checks for monitoring:

```
@router.get("/health")
async def copilot_health():
    """Lightweight health check for copilot routes to verify wiring.

    Returns minimal info without invoking heavy dependencies.
    """
    try:
        registry = _get_predictor_registry()
```

```
    if hasattr(registry, "keys"):
        registered = list(registry.keys())
    else:
        registered = []
except Exception:
    registered = []

return {
    "status": "ok",
    "registered_actions": registered,
    "timestamp": int(time.time()),
}
```

Conclusion

The new architecture provides a clean, orchestrated system with ChatOrchestrator as the single source of truth for the chat response lifecycle. The architecture offers clear separation of concerns, improved testability, enhanced error handling, and transactional integrity for memory operations.

The request flow is well-defined and monitored at every stage, with comprehensive error handling and fallback mechanisms to ensure system reliability. The architecture is designed to be scalable, extensible, and maintainable, making it suitable for production deployment.